

---

C U R S O   D E   A D M I N I S T R A C I Ó N   D E   K A F K A

## LABORATORIO 08b

# Kafka sin Docker adelante

*Instalación sobre RHEL 9 con yum.*

*Guía para hacerlo por tu cuenta, paso a paso*

Confluent Platform 8.2.0 · RHEL 9 (UBI) · un solo nodo

Duración estimada · 45 minutos

NovaTech Logistics · caso de estudio

# Antes de empezar

---

## Qué vas a lograr

En los laboratorios anteriores ejecutaste comandos reales de Kafka contra clústeres reales. Nada de eso fue una simulación. **Pero hubo una capa que no viste nunca, porque Docker la tapa entera.**

Hoy vas a instalar Kafka con `yum` en un servidor RHEL 9, a editar su `server.properties` con un editor de texto, a arrancarlo con `systemd`, y a terminar ejecutando **los mismos comandos de los laboratorios 01 y 02, sin `docker exec` adelante.**

### LO QUE ESTE LABORATORIO DEMUESTRA

*Los cuarenta comandos son exactamente los mismos. Lo único que cambia es la capa de abajo.*

**Si al final dices «es lo mismo, pero sin Docker adelante», el laboratorio cumplió.**

## Tres cosas dichas de frente antes de empezar

### UNO · EL SISTEMA ES RHEL 9 DE VERDAD

La imagen base es **UBI 9** (*Universal Base Image*), la imagen oficial de Red Hat, libre y sin suscripción. No es Rocky ni AlmaLinux, que son compatibles pero no son RHEL. **Es RHEL**, y lo vas a comprobar tú mismo.

### DOS · INSTALAMOS EL BROKER, NO LA PLATAFORMA COMPLETA

En producción SUNAT instala `confluent-platform`, que arrastra Schema Registry, ksqlDB, Connect y Control Center. Son unos 2,4 GB para un laboratorio de un solo broker.

Aquí instalamos `confluent-kafka`, el broker empaquetado por Confluent, que trae **exactamente los mismos comandos**. El procedimiento, o sea el repositorio, la clave GPG y el `yum install`, **es idéntico al de producción**. Lo único que cambia es el nombre del paquete.

## TRES · SYSTEMD CORRE DENTRO DE UN CONTENEDOR

Este laboratorio monta el árbol de cgroups del sistema para que `systemd` pueda arrancar dentro del contenedor. Es lo mínimo que necesita. **No usa `--privileged` y no agrega ninguna capacidad**, así que el contenedor tiene los mismos permisos que cualquier otro lab del curso. En un servidor real esto no hace falta, porque allí `systemd` es el sistema.

## Qué necesitas tener listo

| Requisito                                | Cómo lo compruebas                                                                           |
|------------------------------------------|----------------------------------------------------------------------------------------------|
| Docker Desktop corriendo                 | Solo para levantar el servidor. Después no lo vuelves a usar                                 |
| Conexión a internet                      | La primera vez, el servidor se construye descargando el paquete del repositorio de Confluent |
| Git Bash abierto                         | <code>cd Capitulo_3/lab-08b-instalacion-rhel</code>                                          |
| Haber hecho los laboratorios 01, 02 y 03 | Hoy se comparan sus salidas con las de aquí, línea por línea                                 |

## LA METÁFORA · HOY ABRES EL LOCAL VACÍO

En los trece laboratorios anteriores trabajaste en **un restaurante llave en mano**. Llegabas, encendías las luces y ya estaba todo. La cocina instalada, el gas conectado, el personal contratado.

Hoy **abres el local vacío**. Hay que contratar el suministro, que es el **repositorio**. Instalar la cocina, que es el **paquete**. Escribir en el reglamento a qué hora se enciende, que es el **servicio de systemd**. Y decir **quién tiene la llave**, que es el usuario del sistema.

**Y la carta es exactamente la misma**. Los mismos platos, los mismos nombres, la misma preparación. Esa es la afirmación del laboratorio.

# Entrar al servidor

## Qué vamos a hacer

Levantar el servidor RHEL y entrar en él. Este paso tiene el único `docker` de todo el laboratorio.

## Los comandos

```
cd Capitulo_3/lab-08b-instalacion-rhel
chmod +x bin/*.sh
bin/start-lab.sh
```

`bin/start-lab.sh` — construye la imagen de RHEL 9 con Kafka instalado por `yum`, levanta el contenedor y espera a que `systemd` termine de arrancar. **La primera vez tarda varios minutos**, porque descarga y compila la imagen. Las siguientes son inmediatas

### EL FINAL DE LO QUE VAS A VER · RECORTADO

✓ systemd operativo: running

Sistema: RHEL 9 (UBI)  
Paquete: confluent-kafka-8.2.0-1.noarch  
Comandos: 40 binarios kafka-\* en /usr/bin  
Servicio: confluent-kafka esta inactive

El broker todavia NO esta encendido, y eso es a proposito:  
formatear el almacenamiento y arrancar el servicio es el lab.

Entra al servidor:  
`docker exec -it kafka-rhel bash`

## Ahora entra

```
docker exec -it kafka-rhel bash
```

`docker exec -it` — la `-i` mantiene la entrada abierta y la `-t` te da una terminal. **Juntas son las que te dan una consola interactiva**

`kafka-rhel` — el nombre del servidor. **Fíjate en que no se llama `kafka-broker-1`**, porque no es un broker de los labs anteriores. Es un servidor

`bash` — el programa que quieres ejecutar adentro, que es una consola

A PARTIR DE AQUÍ, OLVÍDATE DE DOCKER

**Ese fue el único `docker` del laboratorio.** De ahora en adelante estás dentro de un servidor RHEL y todos los comandos son los de siempre. El prompt te lo va a recordar, porque dice `rhel9-kafka`.

#### VAS BIEN SI...

El prompt cambió y ahora nombra a `rhel9-kafka`. Compruébalo:

```
cat /etc/redhat-release
systemctl is-system-running
```

```
Red Hat Enterprise Linux release 9.8 (Plow)
running
```

SI `START-LAB.SH` FALLA CONSTRUYENDO LA IMAGEN

**Casi siempre es la red.** El `yum install` baja el paquete de `packages.confluent.io`. Si estás detrás de un proxy corporativo, puede que no salga.

Vuelve a intentarlo. Docker reutiliza las capas que ya bajó, así que el segundo intento arranca donde se quedó el primero.

SI DICE ALGO SOBRE `CGROUP` O `READ-ONLY FILE SYSTEM`

`systemd` necesita escribir en el árbol de cgroups y no pudo. **No lo arregles con `--privileged`.** Comprueba primero que estás usando Docker Desktop actualizado, y si persiste, avisa antes de cambiar el compose.

## De dónde sale Kafka sin Docker

### Qué vamos a hacer

Sin Docker, Kafka sale de **un repositorio de paquetes**. Vamos a mirar el que quedó declarado en el servidor, porque cada línea de ese archivo importa en producción.

#### CONCEPTO · REPOSITORIO DE PAQUETES

Un **servidor del que tu sistema se baja software**, con una lista de qué hay y una firma que prueba quién lo publicó. Es el equivalente de la tienda de aplicaciones del teléfono. En RHEL se declaran en `/etc/yum.repos.d/`, un archivo por repositorio.

#### CONCEPTO · PAQUETE RPM

La **unidad en que se instala software** en RHEL. Trae los binarios, los archivos de configuración, la definición del servicio y las instrucciones para crear el usuario que lo va a correr. **Todo eso lo hace una sola línea de `yum install`.**

### El comando

```
cat /etc/yum.repos.d/confluent.repo
```

**`/etc/yum.repos.d/`** — la carpeta donde RHEL guarda la lista de repositorios de los que puede instalar.  
**Un archivo por repositorio**

#### LO QUE VAS A VER

```
[Confluent]
name=Confluent repository
baseurl=https://packages.confluent.io/rpm/8.2
gpgcheck=1
gpgkey=https://packages.confluent.io/rpm/8.2/archive.key
enabled=1
```

| Línea                   | Por qué importa en producción                                                                                                                                      |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>baseurl</code>    | La rama 8.2 del repositorio oficial de Confluent. <b>De aquí sale el software</b>                                                                                  |
| <code>gpgkey</code>     | La clave pública con la que Confluent firma sus paquetes                                                                                                           |
| <code>gpgcheck=1</code> | <b>No es decorativo.</b> Obliga a <code>yum</code> a verificar esa firma antes de instalar. Sin él, <code>yum</code> instalaría lo que sea que responda en esa URL |

## Qué quedó instalado

```
rpm -q confluent-kafka
rpm -qi confluent-kafka | grep -E "^(Name|Version|License)"
```

`rpm -q` — pregunta si un paquete está instalado y con qué versión. La `q` es de *query*

`rpm -qi` — la ficha completa del paquete. Aquí la filtramos a tres líneas con `grep`

### LO QUE VAS A VER

```
confluent-kafka-8.2.0-1.noarch
```

```
Name      : confluent-kafka
Version    : 8.2.0
License    : Apache (v2)
```

### SOBRE LA VERSIÓN FIJA

El repositorio sirve toda la rama 8.2 y hoy ya ofrece una versión más nueva que la 8.2.0. **El laboratorio la fija a propósito**, porque es la misma de los otros trece labs, y sin esa igualdad las salidas que vas a comparar en el paso 6 dejarían de calzar. **En producción se hace lo mismo, y por la misma razón.** Un clúster no se actualiza porque el repositorio publicó algo el martes.

## Dónde quedaron los comandos

```
ls /usr/bin/kafka-* | head -8
ls /usr/bin/kafka-* | wc -l
```

### LO QUE VAS A VER

```
/usr/bin/kafka-acls  
/usr/bin/kafka-broker-api-versions  
/usr/bin/kafka-client-metrics  
/usr/bin/kafka-cluster  
/usr/bin/kafka-configs  
/usr/bin/kafka-console-consumer  
/usr/bin/kafka-console-producer  
/usr/bin/kafka-console-share-consumer
```

40

### VAS BIEN SI...

**Cuarenta comandos, en `/usr/bin`, en el PATH.**

Son exactamente los mismos cuarenta que hay dentro de la imagen `cp-kafka:8.2.0` que usaste en los labs 01 al 08. **Mismo nombre, misma cantidad, mismo origen.** Lo que cambió es que aquí no hay ningún contenedor delante. Son binarios del sistema, como `ls` o `grep`.

**Anota una respuesta.** ¿Cuántos de esos 40 comandos usaste ya en el curso?



## Dónde está la configuración de verdad

### Qué vamos a hacer

Este es el paso que conecta con el **laboratorio 03**, y conviene recordar qué pasó allí.

En el lab 03 listaste los `.properties` de un broker en Docker y descubriste algo incómodo. **Había un archivo llamado `server.properties` que no era el que el broker estaba usando.** El que mandaba era `kafka.properties`, y no lo había escrito nadie. **Lo generaba la imagen al arrancar**, traduciendo las variables `KAFKA_*` de tu compose.

### Compruébalo aquí

```
ls /etc/kafka/*.properties | wc -l
```

LO QUE VAS A VER

12

#### DOCE, Y EN DOCKER SON TRECE

En un contenedor de los labs anteriores **que esté corriendo**, ese mismo comando devuelve **13**. El decimotercero es `kafka.properties`, y **solo existe cuando el contenedor está encendido**. En la imagen apagada tampoco está, y ahí también son 12.

**Aquí no aparece nunca, porque nadie lo genera.**

### Y ahora la diferencia que importa

```
grep ExecStart /usr/lib/systemd/system/confluent-kafka.service
```

**/usr/lib/systemd/system/** — donde viven las definiciones de los servicios que instala un paquete  
**ExecStart** — la línea que dice **qué comando ejecuta systemd** al arrancar el servicio

LO QUE VAS A VER

```
ExecStart=/usr/bin/kafka-server-start /etc/kafka/server.properties
```

## COMPÁRALO CON LO QUE VEÍAS EN DOCKER

En Docker el proceso arrancaba así:

```
java ... kafka.Kafka /etc/kafka/kafka.properties <- el generado
```

Aquí arranca con `server.properties`, el archivo que vas a editar tú en el paso siguiente. No hay traducción, no hay generación, no hay variable de entorno intermedia. **Lo que escribes es lo que lee el broker.**

## Míralo antes de tocarlo

```
grep -vE "^#|^$" /etc/kafka/server.properties | head -12
```

`grep -vE "^#|^$" —` la `-v` invierte el filtro, así que **quita** las líneas que empiezan por `#`, que son comentarios, y las vacías. Sin esto el archivo son cientos de líneas casi todas comentadas

## LO QUE VAS A VER

```
process.roles=broker,controller
node.id=1
controller.quorum.bootstrap.servers=localhost:9093
listeners=PLAINTEXT://:9092,CONTROLLER://:9093
inter.broker.listener.name=PLAINTEXT
advertised.listeners=PLAINTEXT://localhost:9092,CONTROLLER://localhost:9093
controller.listener.names=CONTROLLER
listener.security.protocol.map=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,SSL:SSL,...
num.network.threads=3
num.io.threads=8
socket.send.buffer.bytes=102400
socket.receive.buffer.bytes=102400
```

### VAS BIEN SI...

**Reconoces todo.** `process.roles=broker,controller` es el nodo combinado del laboratorio 01. El `node.id`, los `listeners`, el `controller.listener.names`, todo eso lo escribiste como variables `KAFKA_*` en un compose.

**Aquí ya viene escrito, en su forma nativa, en un archivo de texto.**

## Editar el archivo y formatear el disco

### Qué vamos a hacer

Dos cambios en el `server.properties`, con un editor de texto. **Esto es lo que Docker no te dejaba hacer nunca.**

```
vi /etc/kafka/server.properties
```

#### SI NUNCA USASTE VI · LO MÍNIMO PARA SOBREVIVIR

| Para                              | Teclas                                              |
|-----------------------------------|-----------------------------------------------------|
| Buscar una línea                  | <code>/</code> y el texto, después Enter            |
| Empezar a escribir                | <code>i</code>                                      |
| Dejar de escribir                 | <code>Esc</code>                                    |
| Guardar y salir                   | <code>:wq</code> y Enter                            |
| Salir sin guardar, si te perdiste | <code>Esc</code> , después <code>:q!</code> y Enter |

### Cambio 1 · el quórum, en la forma del laboratorio 02

Busca esta línea:

```
controller.quorum.bootstrap.servers=localhost:9093
```

Y déjala así:

```
controller.quorum.voters=1@localhost:9093
```

**Es literalmente la propiedad que analizaste en el laboratorio 02**, con su formato `id@servidor:puerto`. Allí eran tres votantes y aquí es uno, porque este servidor está solo. **Escribirla a mano, en el archivo real, es distinto a verla generada.**

## Cambio 2 · dónde viven los datos

Busca:

```
log.dirs=/tmp/kraft-combined-logs
```

Y déjala así:

```
log.dirs=/var/lib/kafka/data
```

`/tmp` se borra al reiniciar. `/var/lib/kafka/data` es la ruta que usaron todos los laboratorios anteriores, y en este servidor está sobre un disco que persiste. **Es la misma trampa que viste en el laboratorio 01**, ahora en el archivo de verdad.

## Verifica tus cambios

```
grep -E "(process.roles|node.id|controller.quorum.voters|log.dirs)" \
/etc/kafka/server.properties
```

LO QUE TIENE QUE SALIR

```
process.roles=broker,controller
node.id=1
controller.quorum.voters=1@localhost:9093
log.dirs=/var/lib/kafka/data
```

### VAS BIEN SI...

Salen **exactamente cuatro líneas**, y las dos que editaste dicen lo que tienen que decir.

Si `controller.quorum.voters` **no** aparece, es que dejaste la línea vieja con su nombre original. Vuelve al editor.

## Ahora formatea el almacenamiento

Es **el mismo comando del laboratorio 01**. Compáralo mentalmente con el que corriste allí.

```
runuser -u cp-kafka -- kafka-storage format \
--cluster-id MkU3OEVbNTcwNTJENDM2Qk \
--config /etc/kafka/server.properties
```

**`runuser -u cp-kafka --`** — ejecuta el comando como el usuario del servicio, no como root. Si formateas como root, los archivos quedan de root y el servicio, que corre como `cp-kafka`, no va a poder

escribir. Es el error de permisos más común al instalar Kafka a mano

`kafka-storage format` — graba la identidad del clúster en el disco. Lo mismo que la imagen de Confluent hacía sola en el laboratorio 01

`--cluster-id` — la identidad. Aquí va fija para que puedas comparar salidas con los otros labs

`--config /etc/kafka/server.properties` — el archivo que acabas de editar. En Docker este argumento venía cocinado

LO QUE VAS A VER · LA PRIMERA LÍNEA ES LARGA Y NO IMPORTA

```
Bootstrap metadata: BootstrapMetadata(records=[...metadata.version...])
Formatting metadata directory /var/lib/kafka/data with metadata.version 4.2-IV1.
```

#### VAS BIEN SI...

La segunda línea dice `Formatting metadata directory /var/lib/kafka/data`. Esa ruta es la que tú escribiste, y es la prueba de que el archivo que editaste es el que se está leyendo.

#### SI DICE QUE EL DIRECTORIO YA ESTÁ FORMATEADO

Ya lo hiciste antes, o estás repitiendo el laboratorio. No es un problema, sigue al paso 5. Si quieres empezar de cero de verdad, sal del servidor con `exit` y ejecuta `bin/reset-lab.sh`.

#### SI DICE `PERMISSION DENIED`

Se te olvidó el `runuser -u cp-kafka --` del principio, o el directorio quedó de root de un intento anterior. Arréglalo con `chown -R cp-kafka:confluent /var/lib/kafka/data` y repite.

## Arrancar el servicio

### El comando

```
systemctl start confluent-kafka
systemctl status confluent-kafka
```

**systemctl start** — arranca el servicio **ahora**

**systemctl status** — muestra cómo está. **Se sale con q** si te deja en un paginador

#### LO QUE VAS A VER · RECORTADO

```
• confluent-kafka.service - Apache Kafka - broker
   Loaded: loaded (/usr/lib/systemd/system/confluent-kafka.service;
           disabled; preset: disabled)
   Active: active (running) since Thu 2026-08-27 17:02:58 UTC; 12s ago
     Docs: http://docs.confluent.io/
   Main PID: 416 (java)
    Tasks: 122 (limit: 79998)
   Memory: 389.7M (peak: 399.4M)
      CPU: 4.055s
```

#### CONCEPTO · UNIDAD DE SYSTEMD

Un **archivo de texto que le dice al sistema cómo se enciende un servicio**. Qué comando ejecutar, con qué usuario, qué hacer si se muere y cuánto esperarle al apagar. Lo instala el paquete, y vive en `/usr/lib/systemd/system/`. **En Docker su equivalente era el `entrypoint` de la imagen**, que no se puede leer tan fácil.

### Lee la unidad completa, que es corta y lo dice todo

```
cat /usr/lib/systemd/system/confluent-kafka.service
```

#### LO QUE VAS A VER

```
[Unit]
Description=Apache Kafka - broker
Documentation=http://docs.confluent.io/
After=network.target

[Service]
Type=simple
User=cp-kafka
Group=confluent
ExecStart=/usr/bin/kafka-server-start /etc/kafka/server.properties
LimitNOFILE=1000000
TimeoutStopSec=180
Restart=no

[Install]
WantedBy=multi-user.target
```

| Línea                            | Qué significa, y por qué se revisa en producción                                                                                                          |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>User=cp-kafka</code>       | <b>El broker no corre como root.</b> Si alguien lo compromete, no se lleva el servidor                                                                    |
| <code>LimitNOFILE=1000000</code> | Kafka abre un descriptor de archivo por segmento y por conexión. <b>Con el límite por defecto del sistema, un broker con carga real se cae</b>            |
| <code>TimeoutStopSec=180</code>  | <code>systemd</code> le da tres minutos para cerrar ordenado antes de matarlo. <b>Cerrar ordenado es lo que evita la recuperación de logs al arrancar</b> |
| <code>Restart=no</code>          | <b>Si el broker muere, no se levanta solo.</b> En un clúster eso es deliberado. Prefieres enterarte                                                       |

## Dónde se miran los logs

```
journalctl -u confluent-kafka -n 50
ls /var/log/kafka/
```

`journalctl -u <servicio>` — lo que `systemd` capturó de la salida del proceso

`-n 50` — las últimas cincuenta líneas

### CONCEPTO · EL JOURNAL

El **registro central de todo lo que pasa en el sistema**, que `systemd` mantiene. Cada servicio escribe ahí, y se consulta con `journalctl`. **Es el primer sitio donde se mira cuando un servicio no arranca**, y en el paso 7 vas a ver por qué.

```
controller.log
kafka-authorizer.log
kafka-request.log
kafkaServer-gc.log
log-cleaner.log
server.log
state-change.log
```

#### DOS CAMINOS, Y HAY QUE CONOCER LOS DOS

`journalctl` te da lo que `systemd` capturó. `/var/log/kafka/` son los archivos que escribe el propio Kafka, con mucho más detalle y separados por tema. En Docker esto era `docker logs` y punto, todo mezclado en un solo flujo.

#### VAS BIEN SI...

Active: dice `active (running)` y el `Main PID` apunta a un proceso `java`.

Y en `/var/log/kafka/` hay **siete archivos**, con `server.log` y `controller.log` entre ellos.

#### SI DICE `ACTIVE: FAILED` NADA MÁS ARRANCAR

Mira el journal, que es donde está el diagnóstico:

```
journalctl -u confluent-kafka -n 30 --no-pager
```

**Las dos causas, por frecuencia.** Si menciona permisos o `/var/lib/kafka/data`, formateaste como root. Arréglalo con `chown -R cp-kafka:confluent /var/lib/kafka/data`.

Si menciona `controller.quorum`, la línea que editaste en el paso 4 tiene una errata. Vuelve al archivo.



## Los mismos comandos, sin envase

### Qué vamos a hacer

**Este es el corazón del laboratorio.** Todo lo que sigue lo ejecutaste ya en los laboratorios 01 y 02, con `docker exec` adelante. Ahora no.

### La carta de servicios del broker

```
kafka-broker-api-versions --bootstrap-server localhost:9092
```

sin `docker exec` — el comando está en el PATH, como `ls`

`--bootstrap-server localhost:9092` — aquí `localhost` es de verdad este servidor. En los labs anteriores tenías que usar el nombre del contenedor y el puerto interno

#### LAS PRIMERAS LÍNEAS

```
localhost:9092 (id: 1 rack: null isFenced: false) -> (
  Produce(0): 0 to 13 [usable: 13],
  Fetch(1): 4 to 18 [usable: 18],
  ListOffsets(2): 1 to 11 [usable: 11],
  Metadata(3): 0 to 13 [usable: 13],
  OffsetCommit(8): 2 to 10 [usable: 10],
```

### Crear, listar y describir un tópico

```
kafka-topics --bootstrap-server localhost:9092 \
  --create --topic novatech.lab08b.demo --partitions 3 --replication-factor 1

kafka-topics --bootstrap-server localhost:9092 --list

kafka-topics --bootstrap-server localhost:9092 \
  --describe --topic novatech.lab08b.demo
```

#### LO QUE VAS A VER

```
WARNING: Due to limitations in metric names, topics with a period ('.') or
underscore ('_') could collide. To avoid issues it is best to use either,
but not both.
```

```
Created topic novatech.lab08b.demo.
```

```
novatech.lab08b.demo
```

```
Topic: novatech.lab08b.demo  TopicId: WJJxTYAFR5qJ2gvd72VSAw  PartitionCount: 3
  ReplicationFactor: 1   Configs: min.insync.replicas=1,segment.bytes=1073741824
  Partition: 0  Leader: 1  Replicas: 1  Isr: 1
  Partition: 1  Leader: 1  Replicas: 1  Isr: 1
  Partition: 2  Leader: 1  Replicas: 1  Isr: 1
```

**Factor de replicación 1**, porque este servidor está solo. Es la misma limitación del laboratorio 01, y por la misma razón.

## El quórum

```
kafka-metadata-quorum --bootstrap-server localhost:9092 describe --status
```

### LO QUE VAS A VER

```
ClusterId:          Mku3OEVbNTcwNTJENDM2Qk
LeaderId:           1
LeaderEpoch:       1
HighWatermark:      50
MaxFollowerLag:     0
MaxFollowerLagTimeMs: 0
CurrentVoters:      [{"id": 1, "endpoints": ["CONTROLLER://localhost:9093"]}]}
CurrentObservers:   []
```

Es la misma salida del laboratorio 02, con **un votante en vez de tres**. Y ese

`CONTROLLER://localhost:9093` es la línea que tú escribiste con `vi` en el paso 4.

## Lado a lado

Estos mismos comandos se corrieron contra un broker en Docker, con la configuración del laboratorio 01 y el mismo `--cluster-id`. Este es el resultado.

| Comando                                | Diferencia entre Docker y RHEL                                                                                                                                                                                                                                                                                                        |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>kafka-broker-api-versions</code> | <b>Ninguna. Idénticas byte a byte</b>                                                                                                                                                                                                                                                                                                 |
| <code>kafka-topics --create</code>     | <b>Ninguna. Idénticas byte a byte</b>                                                                                                                                                                                                                                                                                                 |
| <code>kafka-topics --describe</code>   | Solo el <code>TopicId</code> , que Kafka genera al azar en cada tópico                                                                                                                                                                                                                                                                |
| <code>kafka-metadata-quorum</code>     | Solo el <code>HighWatermark</code> , que es un contador que avanza con el tiempo, y el endpoint del controlador. En Docker era <code>CONTROLLER://kafka-broker:39092</code> y aquí es <code>CONTROLLER://localhost:9093</code> . <b>Es el nombre de host y el puerto que eligió cada laboratorio, no una diferencia de plataforma</b> |

#### LO LOGRASTE SI...

**No encuentras ninguna diferencia atribuible a estar sobre RHEL en vez de sobre Docker.** Los comandos, los parámetros y las salidas son los mismos.

Eso es exactamente lo que el curso viene afirmando desde el laboratorio 01, y aquí queda demostrado.

## Lo que Docker escondía

### El usuario y los permisos

```
ps -o user=,pid=,comm= -C java
id cp-kafka
ls -ld /var/lib/kafka/data
```

`ps -C java` — busca los procesos que se llaman `java`

`-o user=,pid=,comm=` — muestra solo tres columnas, y el `=` al final de cada una quita el encabezado

`id <usuario>` — a qué grupos pertenece y con qué números

`ls -ld` — la `-d` muestra el **directorio en sí**, no lo que hay dentro

#### LO QUE VAS A VER

```
cp-kafka    416 java
uid=998(cp-kafka) gid=998(confluent) groups=998(confluent)
drwxr-xr-x 6 cp-kafka confluent 4096 Aug 27 17:03 /var/lib/kafka/data
```

#### POR QUÉ ESTO ES LO PRIMERO QUE SE REVISAS

El paquete creó el usuario `cp-kafka` y el grupo `confluent` al instalarse. El broker corre con ese usuario y el directorio de datos le pertenece. **En producción esto es lo primero que se mira cuando un broker no arranca**, porque casi siempre alguien formateó o copió archivos como root.

### Dónde están los datos, en un disco de verdad

```
ls /var/lib/kafka/data
df -h /var/lib/kafka/data
```

#### LO QUE VAS A VER

```
__cluster_metadata-0
bootstrap.checkpoint
cleaner-offset-checkpoint
log-start-offset-checkpoint
meta.properties
novatech.lab08b.demo-0
novatech.lab08b.demo-1
novatech.lab08b.demo-2
recovery-point-offset-checkpoint
replication-offset-checkpoint

/dev/vda1          911G   64G   801G   8% /var/lib/kafka/data
```

Un directorio por partición, y ahí están las tres del tópico que creaste. Y `__cluster_metadata-0`, que es el log de KRaft de los laboratorios 01 y 02.

Kafka se queda sin disco antes que sin memoria, y ese `df` es la línea que se vigila en un servidor real.

## Y ahora el `failed` que no es un fallo

Apaga el servicio de forma ordenada, tú mismo, y mira el estado:

```
systemctl stop confluent-kafka
systemctl is-active confluent-kafka
```

LO QUE VAS A VER

```
failed
```

`FAILED`, DESPUÉS DE UN APAGADO ORDENADO QUE TÚ PEDISTE

No es un error tuyo ni del laboratorio. Es la unidad tal como la publica Confluent.

`kafka-server-start` termina con un código distinto de cero cuando recibe la señal de apagado, la unidad no declara `SuccessExitStatus`, y `systemd` lo anota como fallo. En un servidor de SUNAT pasa exactamente igual.

## La prueba de que el apagado fue limpio

La da el propio Kafka la próxima vez que arranca:

```
systemctl start confluent-kafka
journalctl -u confluent-kafka | grep "clean shutdown"
```

## LO QUE VAS A VER

```
Skipping recovery of 3 logs from /var/lib/kafka/data since clean shutdown  
file was found (kafka.log.LogManager)
```

Kafka encontró la marca de apagado limpio y **se saltó la recuperación de los logs**. Si el apagado hubiera sido sucio, aquí diría que está recuperando, y tardaría.

### LA LECCIÓN, Y VALE PARA CUALQUIER SERVICIO

**Una palabra en `systemctl` no es un diagnóstico. El diagnóstico está en el journal.**

## Y una última, que se confunde todo el tiempo

```
systemctl is-enabled confluent-kafka
```

## LO QUE VAS A VER

```
disabled
```

`start` enciende el servicio **ahora**. `enable` hace que arranque **al encender el servidor**. Son cosas distintas. **Si solo hiciste `start`, el día que ese servidor se reinicie el broker no vuelve.**

```
systemctl enable confluent-kafka
```

### LO LOGRASTE SI...

Viste el `failed` tras un apagado que tú pediste, encontraste la línea de `clean shutdown` en el journal, y entiendes por qué `is-enabled` decía `disabled`.

**Anota una respuesta.** En tu clúster de SUNAT, ¿las unidades de Kafka están `enabled`?

# Lo que aprendiste

---

## 1 Los cuarenta comandos son los mismos.

Todo lo que aprendiste en trece laboratorios se aplica igual cuando le quitas el envase. Lo que cambia es la capa de abajo, o sea el paquete, el servicio, el archivo y el usuario. Y ésa es justamente la capa que administras en SUNAT.

## 2 En Docker no editas la configuración. En un servidor, sí.

La imagen traduce variables `KAFKA_*` a un `kafka.properties` que genera al arrancar. En RHEL el archivo es `server.properties`, lo escribes tú, y es el que el proceso lee. Sin intermediarios.

## 3 El usuario del servicio es lo primero que se revisa.

El broker corre como `cp-kafka`, no como root. Si formateas o copias archivos como root, el servicio no puede escribir y no arranca. Es el error de instalación más común.

## 4 Una palabra en `systemctl` no es un diagnóstico.

`failed` tras un apagado ordenado es normal en esta unidad. Y `enabled` y `active` son cosas distintas. La respuesta siempre está en el journal, no en la palabra.

# Para profundizar

## A · Comprobar que es RHEL de verdad

```
cat /etc/redhat-release
head -3 /etc/os-release
```

LO QUE SALE

```
Red Hat Enterprise Linux release 9.8 (Plow)

NAME="Red Hat Enterprise Linux"
VERSION="9.8 (Plow)"
ID="rhel"
```

**Lo que hay que mirar.** Ese `ID="rhel"` es el que no tendría una distribución compatible. Rocky diría `ID="rocky"` y Alma diría `ID="almalinux"`. **Es la imagen oficial de Red Hat**, libre y sin suscripción.

## B · Contar los comandos en los dos mundos

Sal del servidor con `exit` y compara desde tu máquina:

```
docker exec kafka-rhel sh -c 'ls /usr/bin/kafka-* | wc -l'
docker run --rm confluentinc/cp-kafka:8.2.0 sh -c 'ls /usr/bin/kafka-* | wc -l'
```

LO QUE SALE

```
40
40
```

**Lo que hay que mirar.** El mismo número, desde dos plataformas distintas. **Es la afirmación del laboratorio, en dos líneas.**

## C · El decimotercer archivo, comprobado

Es la parte del paso 3 que aquí no se puede ver, porque hace falta un contenedor de los otros labs corriendo. Levanta el laboratorio 02 y prueba:

```
docker exec kafka-broker-1 sh -c 'ls /etc/kafka/*.properties | wc -l'
docker run --rm confluentinc/cp-kafka:8.2.0 sh -c 'ls /etc/kafka/*.properties | wc -l'
docker exec kafka-rhel sh -c 'ls /etc/kafka/*.properties | wc -l'
```

LO QUE SALE



```
13     <- contenedor corriendo
12     <- la misma imagen, apagada
12     <- el servidor RHEL
```

Y para ver cuál usa de verdad el contenedor:

```
docker exec kafka-broker-1 sh -c "ps -o args= -C java | tr ' ' '\n' | grep properties"
```

LO QUE SALE

```
/etc/kafka/kafka.properties
```

**Lo que hay que mirar.** El decimotercero es `kafka.properties`, **solo existe mientras el contenedor corre, y es el único que el proceso lee.** Ese archivo no existe en RHEL porque allí nadie lo genera.

## D · Qué pasa si formateas como root

Es el error más común, y vale la pena provocarlo a propósito para reconocerlo:

```
systemctl stop confluent-kafka
rm -rf /var/lib/kafka/data/*
kafka-storage format --cluster-id Mku30EVBNTcwNTJENDM2Qk \
  --config /etc/kafka/server.properties
ls -ld /var/lib/kafka/data/*
systemctl start confluent-kafka
systemctl is-active confluent-kafka
```

**sin `runuser` — ahí está la trampa.** El formateo lo hace root, así que los archivos quedan de root

**Lo que hay que mirar.** El `ls -ld` muestra los archivos con dueño `root`, y el servicio no arranca. El journal lo dice. Para arreglarlo:

```
chown -R cp-kafka:confluent /var/lib/kafka/data
systemctl start confluent-kafka
```

## E · Los logs por tema

```
tail -20 /var/log/kafka/server.log
tail -20 /var/log/kafka/controller.log
```

**Lo que hay que mirar.** Son dos historias distintas del mismo arranque. `server.log` cuenta lo que hizo el broker y `controller.log` lo que hizo el controlador. **En Docker las dos venían mezcladas**

en `docker logs` , y separarlas es media hora menos de diagnóstico el día que algo falle.

## Antes de cerrar

### DEJA EL TERRENO LIMPIO

```
exit                # salir del servidor RHEL
bin/90-test-lab.sh  # ver el estado del laboratorio
bin/stop-lab.sh     # apagar conservando tus datos y tu configuración
```

Si vuelves con `bin/start-lab.sh` , el servidor se reanuda **tal como lo dejaste**, y tu `server.properties` editado sigue ahí. Para volver al servidor recién instalado y repetir el laboratorio desde cero, usa `bin/reset-lab.sh` .

### LO QUE TE LLEVAS

Todo lo que aprendiste en trece laboratorios se aplica igual cuando le quitas el envase. **Lo que cambia es el paquete, el servicio, el archivo y el usuario.** Y ésta es, justamente, la capa que administras en SUNAT.